Introduction

Name: Shreya Patra

Registration No.: 26BCY10043

Email ID: shreya.26bcy10043@vitbhopal.ac.in

Expense Tracker & Budget Manager

A Command-Line Personal Finance Application

Evaluated Course Project — Python Essentials

Submission Date: September 2026

Expense Tracker & Budget Manager

A Command-Line Personal Finance Application

Evaluated Course Project — Python Essentials
Submission Date: September 2026

2. Introduction

This report documents the design, implementation, and evaluation of the **Expense Tracker & Budget Manager**, a command-line application built in Python as the evaluated project for the Python Essentials course. The tool applies core course concepts — functions, object-oriented design, exception handling, file I/O, and modular program structure — to a practical, everyday problem: keeping track of personal spending.

3. Problem Statement

Many individuals track personal expenses informally, in notebooks or ad-hoc spreadsheets, which offer no validation and make it difficult to see spending patterns, enforce a budget, or catch entry mistakes. There is a need for a lightweight, dependency-free tool that a user can run from the terminal to log expenses, set category budgets, and immediately see where their money is going — without setting up a database server or a graphical application.

4. Functional Requirements

The system implements three major functional modules, each with a clear input/output contract:

- **Expense Management** — add, list (filterable by category and month), update, and delete expense records. Input: amount, category, description, date. Output: confirmation message and updated record list.
- **Budget Management** — set or update a monthly spending limit per category; check actual spend against that limit and raise an explicit warning when exceeded. Input: category, monthly limit. Output: budget status per category.
- **Reporting & Analytics** — generate a category-wise spending breakdown, a month-by-month summary, and a top-5 largest-expenses list, each rendered as a text-based bar chart. Input: optional month filter. Output: formatted report printed to the terminal.

User workflow: the application presents a numbered menu; the user selects an option, is prompted for the relevant fields (validated as they are entered), and receives immediate console feedback before returning to the menu.

5. Non-Functional Requirements

Requirement	How it is addressed
Performance	In-memory filtering/aggregation over a local JSON file; operations are O(n), fast for realistic personal da
Security	All user input (amount, category, description, date) is validated for type, range, and length before being
Reliability	Atomic file writes (write to a temp file, then os.replace) prevent data corruption if the program is interrupt
Usability	A simple numbered menu, clear prompts, and plain-language error messages replace raw stack traces.
Maintainability	Code is split into single-responsibility modules (manager, budget, reports, storage) sharing one exceptio

Error handling	A custom exception hierarchy (ValidationError, ExpenseNotFoundError, BudgetExceededError, Storage
Logging / monitoring	All state-changing actions and errors are timestamped and written to app.log, separate from normal con

6. System Architecture

The application follows a layered, modular architecture. The CLI layer is the only component that interacts with the user; it delegates all business logic to three functional modules, which in turn share a single JSON storage layer. This separation means the manager, budget, and reports modules could be reused behind a different interface (e.g. a web API) without modification.

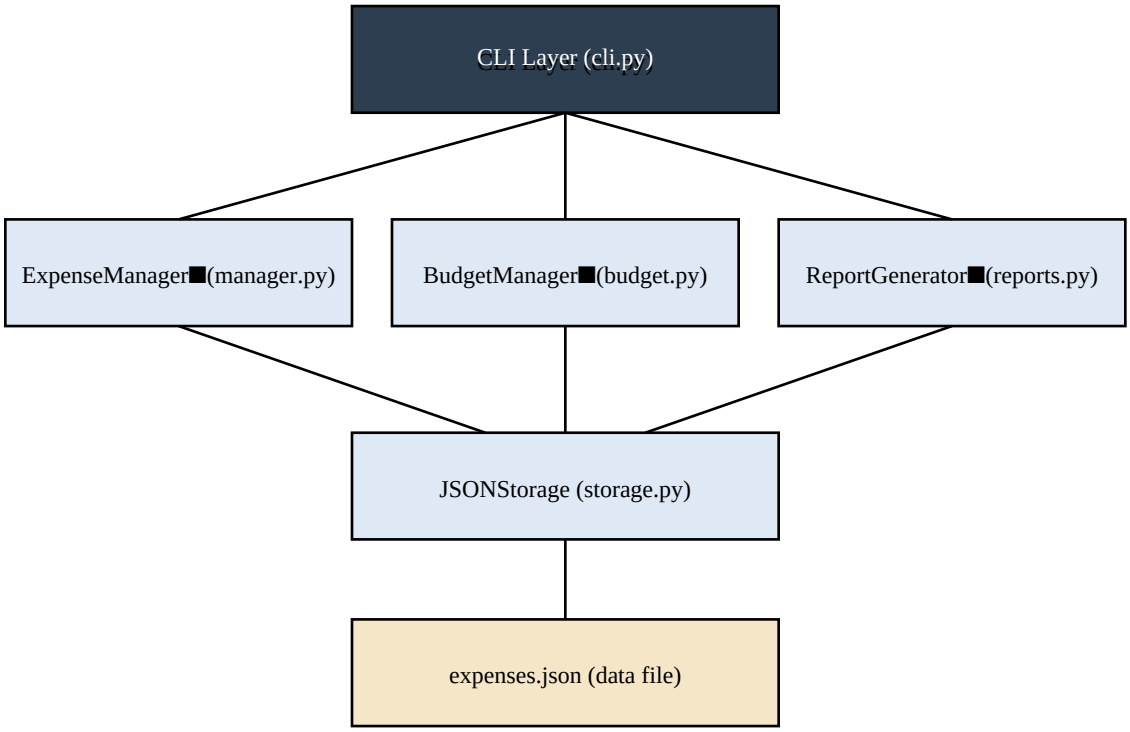
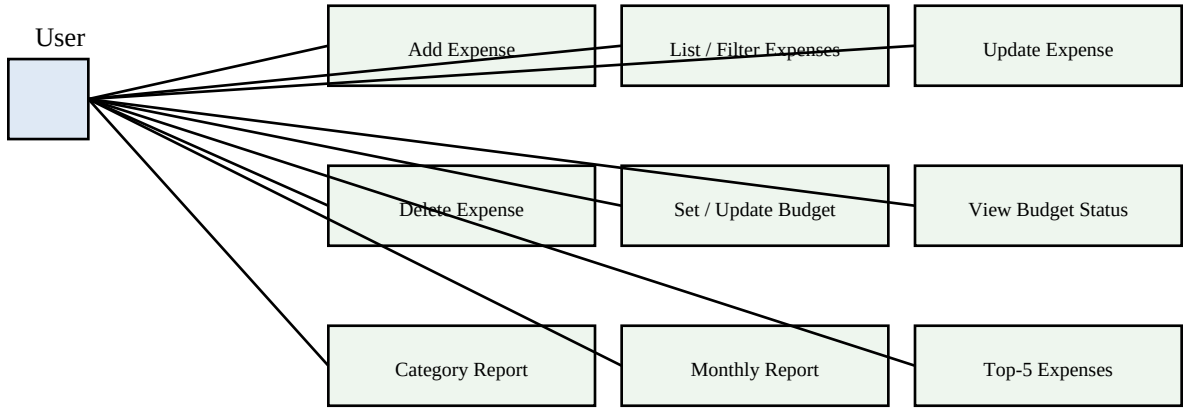


Figure 1: System architecture — the CLI delegates to three functional modules, which share a single JSON storage layer backed by expenses.json.

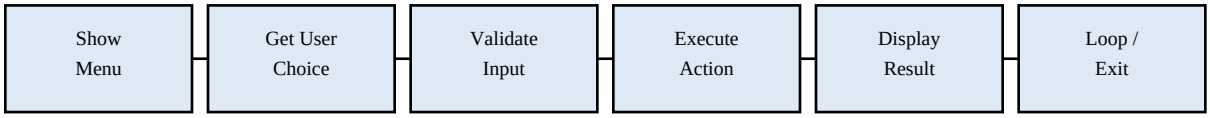
7. Design Diagrams

7.1 Use Case Diagram

Actor: **User**. Use cases: Add Expense, List/Filter Expenses, Update Expense, Delete Expense, Set Budget, View Budget Status, View Category Report, View Monthly Report, View Top Expenses. All use cases are initiated from the single CLI menu; there is no secondary actor since the system is single-user and offline.



7.2 Process Flow / Workflow Diagram

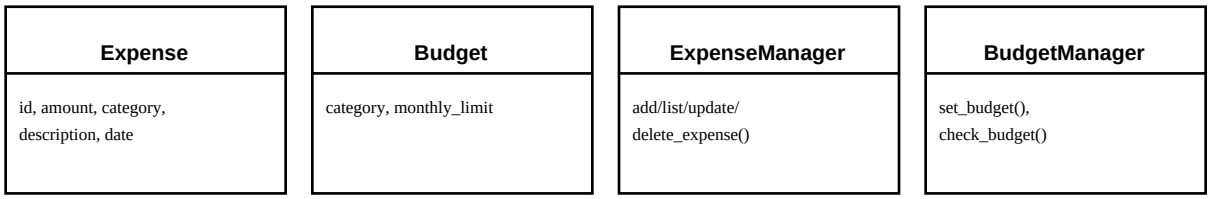


Execute Action covers the CRUD, budget-check, or report-generation logic specific to the option chosen.

7.3 Sequence Diagram (Add Expense flow)

- User selects option 1 (Add Expense) from the CLI menu.
- CLI prompts for amount, category, description, and date.
- CLI calls utils.validate_*() functions on each field.
- On success, CLI calls ExpenseManager.add_expense(...).
- ExpenseManager loads current state via JSONStorage.load(), appends the new record, increments next_id, and calls JSONStorage.save(state).
- CLI calls BudgetManager.check_budget() for the affected category/month.
- CLI prints confirmation, or a budget-exceeded warning if applicable.

7.4 Class / Component Diagram



7.5 Database / Storage Design

The application uses a flat JSON file (`data/expenses.json`) rather than a relational database, appropriate for a single-user local tool. A formal ER diagram does not apply; the equivalent schema is:

Key	Type	Description
expenses	list[object]	Each object: id (int), amount (float), category (str), description (str), date (str, YYYY-MM-DD)
budgets	list[object]	Each object: category (str), monthly_limit (float)
next_id	int	Auto-incrementing counter for new expense IDs

8. Design Decisions & Rationale

- **JSON over a real database:** chosen for zero external dependencies and human-readable data, appropriate for the course's scope and a single-user CLI tool.
- **Dataclasses for models:** Expense and Budget use Python's @dataclass to reduce boilerplate while keeping type hints explicit, directly applying object-oriented concepts from the course.
- **Custom exception hierarchy:** rather than propagating built-in exceptions, a dedicated hierarchy (ValidationError, ExpenseNotFoundError, BudgetExceededError, StorageError) makes the CLI's error handling explicit and demonstrates exception design, not just exception handling.
- **Atomic writes:** writing to a temp file and using os.replace() avoids partial/corrupted writes if the program is killed mid-save.
- **No GUI:** per the project's technical constraints, the entire interface is a terminal menu loop, keeping the tool scriptable and dependency-free.

9. Implementation Details

The codebase is organized as a Python package (`expense_tracker`) with 8 modules plus a top-level entry point and 3 test modules — 12 files in total, exceeding the minimum of 5–10 meaningful files. Key implementation points:

- **storage.py** centralizes all file I/O so no other module touches the filesystem directly, simplifying unit testing (tests use a temporary JSON file).
- **manager.py, budget.py, reports.py** each expose a small, focused public API and depend only on storage.py and models.py — no circular imports.
- **utils.py** centralizes validation logic so the same rules apply whether data comes from the CLI or (in future) another interface.
- **cli.py** is the only module that calls input()/print(), keeping the rest of the application UI-agnostic and independently testable.
- **logging** is configured once in cli.py and used across modules via module-level loggers (e.g. logging.getLogger('expense_tracker.manager')).

10. Screenshots / Results

Sample console session (menu and an Add Expense action):

```
==================== EXPENSE TRACKER ====================
1. Add expense
2. List expenses
3. Update expense
4. Delete expense
5. Set / update a category budget
6. View budget status (current month)
7. Category spending report
8. Monthly summary report
9. Top 5 expenses
0. Exit
=========================================================
Choose an option: 1
Categories: Food, Transport, Housing, Utilities, Entertainment, Health, Shopping, Education,
Other
```

```
Amount: 250
Category: Food
Description: Groceries at market
Date (YYYY-MM-DD, blank = today):
Added: [ 1] 2026-09-26 Food 250.00 Groceries at market
```

Sample category report output:

```
Food | ######################################## 250.00
Transport | ############ 80.00
```

11. Testing Approach

The project uses Python's built-in `unittest` framework with 17 automated tests spread across the three functional modules, each using an isolated temporary JSON file so tests never touch real user data:

- **test_manager.py** (7 tests) — ID assignment, category/month filtering, not-found handling, update, delete, and total calculation.
- **test_budget.py** (5 tests) — budget creation/update, over-budget detection, within-budget pass-through, and multi-category status checks.
- **test_reports.py** (5 tests) — category breakdown totals, monthly grouping, top-expense ordering, and bar-chart rendering (including the empty-data case).

Run with: `python3 -m unittest discover -s tests -v` — all 17 tests pass. Manual end-to-end testing was also performed by piping a scripted sequence of menu choices into the CLI to confirm the full add → budget → report workflow.

12. Challenges Faced

- **Avoiding circular imports** between manager.py and budget.py, since budgets need expense totals — resolved by having BudgetManager accept an ExpenseManager instance rather than importing it the other way.
- **Preventing data corruption** on unexpected interruption during a save — resolved with an atomic write-then-rename pattern in storage.py.
- **Keeping the CLI testable** despite being interactive — resolved by isolating all input()/print() calls in cli.py so the business-logic modules can be unit-tested without any console interaction.

13. Learnings & Key Takeaways

- Designing a custom exception hierarchy makes error handling far more readable than relying on generic exceptions.
- Separating persistence (storage.py) from business logic (manager.py, budget.py, reports.py) makes unit testing dramatically simpler.
- Small, well-named modules with a single responsibility are easier to reason about and extend than one large script.
- Writing tests alongside each module (rather than after the whole project) caught several edge cases early, such as filtering by a month with no expenses.

14. Future Enhancements

- Export reports to CSV or PDF for sharing.
- Support multiple currencies with conversion.
- Add recurring/scheduled expenses (e.g. monthly rent).
- Optionally swap the JSON storage layer for SQLite for larger datasets, without changing the manager/budget/reports modules (thanks to the storage abstraction).

15. References

- Python Software Foundation, *Python 3 Documentation* — json, dataclasses, unittest, logging modules.
- Python Essentials course materials (functions, OOP, exception handling, file I/O).